

Interim Report — Intelligent Complaint Analysis for Financial Services (CrediTrust)

Building a RAG-Powered Chatbot to Turn Customer Feedback into Actionable Insights

Author: Yomiyu Wakweya Repository: <https://github.com/yomiyu15/rag-complaint-chatbot> Covers:

Task 1 (complete) and **Task 2 (complete)**

Environment note: The ML stack initially failed on this machine (Python 3.14 + a missing Visual C++ runtime broke `torch / onnxruntime`). This was resolved by installing the MS VC++ Redistributable and building a **Python 3.12 virtualenv** (`.venv/`). ChromaDB could not be installed reliably over an unstable network, so the vector store uses **FAISS** — which the challenge explicitly permits as an alternative to ChromaDB.

1. Introduction — The Business Problem

CrediTrust Financial is a mobile-first digital finance company serving East African markets with credit cards, personal loans, savings accounts, and money transfers. With 500,000+ users across three countries, it receives thousands of complaints per month via in-app channels, email, and regulatory portals. Product, Support, and Compliance teams cannot read this volume manually, so trends surface slowly and reactively.

The goal is an internal **Retrieval-Augmented Generation (RAG)** chatbot that lets a non-technical stakeholder (e.g. Asha, a Credit Cards PM) ask a plain-English question — "Why are people unhappy with credit cards?" — and receive a synthesized, evidence-backed answer in seconds, with the source complaint excerpts shown for trust. Success is measured by (1) cutting trend-identification time from days to minutes, (2) empowering non-technical teams without a data analyst, and (3) shifting the company from reactive to proactive issue detection.

2. Task 1 — Exploratory Data Analysis & Preprocessing

2.1 Data source

The [Consumer Financial Protection Bureau \(CFPB\)](#) consumer-complaint database — a public, real-world dataset. The raw export is ~6 GB uncompressed with **18 columns** (product/company metadata, a short issue label, a free-text consumer narrative, and submission dates).

Because the file cannot fit in notebook memory, the full pass runs as a streaming script, `src/eda_preprocessing.py`, which writes `data/filtered_complaints.csv`, `reports/eda_summary.json`,

and the figures in `reports/figures/`. The notebook `notebooks/01_eda_preprocessing.ipynb` demonstrates the logic on a 200k-row sample and loads those artifacts.

2.2 Key EDA findings (2–3 paragraph summary)

The full CFPB export holds **9,609,797** complaints, but only **2,980,756 (31.0%)** include a free-text consumer narrative; the remaining **6,629,041** are metadata-only. Since the RAG system depends entirely on narrative text for semantic retrieval, the ~6.6M narrative-less records are unusable and dropped. The corpus is also heavily skewed by product: credit-reporting complaints dominate the raw dataset by an order of magnitude, while our four target families sit further down CFPB's overlapping current/legacy product labels.

After mapping those raw labels to the four target categories and keeping only rows with a narrative, **480,576** complaints remain, distributed as: **Credit Card 189,334**, **Savings Account 155,204**, **Money Transfer 98,701**, and **Personal Loan 37,341**. Personal Loan is clearly the minority class — a fact that directly drives the *stratified* sampling plan in Task 2 so it is not drowned out.

Narrative length varies widely: word counts run from a minimum of 1 to a maximum of 6,469, with a **median of 137** and a **mean of 205.6** (percentiles [1, 25, 50, 75, 95, 99] = [13, 82, 137, 256, 588, 1046]). Most narratives are short-to-medium, but a long tail of highly detailed complaints (**93,443** entries exceed 300 words) makes a single-vector embedding lossy — the primary motivation for the chunking strategy in Task 2. At the other extreme, **272** entries have fewer than 5 words and are effectively noise.

Metric	Value
Total complaints (raw)	9,609,797
With narrative	2,980,756 (31.0%)
Without narrative	6,629,041
Filtered corpus written	480,576
Narrative word count (min / median / mean / max)	1 / 137 / 205.6 / 6,469
Very short (<5 words)	272
Very long (>300 words)	93,443

2.3 Filtering & product mapping

CFPB uses many overlapping product labels. The four target families are assembled via a `PRODUCT_MAP` from raw labels such as "*Credit card or prepaid card*", "*Checking or savings account*", and "*Money transfer, virtual currency, or money service*". Only rows in these four families **and** carrying a non-empty narrative are retained.

2.4 Text cleaning

`clean_narrative()` lowercases text, removes CFPB xxxx PII redactions, strips boilerplate openers (e.g. "I am writing to file a complaint..."), removes special characters, and normalizes whitespace — producing a `cleaned_narrative` column.

2.5 Task 1 deliverables

- [x] EDA + preprocessing code: `src/eda_preprocessing.py`, `notebooks/01_eda_preprocessing.ipynb`
- [x] EDA summary (this section) with real figures
- [x] `data/filtered_complaints.csv` (576 MB — regenerated locally, git-ignored; not pushed as it exceeds GitHub's 100 MB limit)
- [x] Figures: product distribution, narrative availability, category counts, word-count histogram

3. Task 2 — Chunking, Embedding & Vector-Store Indexing

Implemented in `src/chunk_embed_index.py`; run metrics in `reports/task2_summary.json`.

3.1 Stratified sampling

From the **480,576**-row cleaned corpus we drew a **12,000**-complaint sample (`--sample-size 12000 --seed 42`), allocating the quota to each product category **in proportion to its share of the full corpus**. This preserves the real class balance while keeping the minority Personal Loan class (7.8%) represented rather than swamped. The sampled distribution matches the full corpus almost exactly:

Category	Full corpus	Share	Sample
Credit Card	189,333	39.4%	4,728
Savings Account	155,202	32.3%	3,875
Money Transfer	98,700	20.5%	2,465
Personal Loan	37,341	7.8%	932
Total	480,576	100%	12,000

A fixed seed (42) makes the sample fully reproducible.

3.2 Chunking

We used LangChain's `RecursiveCharacterTextSplitter` with `chunk_size=500`, `chunk_overlap=50` (character-based), splitting on paragraph → line → sentence → word boundaries. Rationale: ~19% of narratives exceed 300 words, and embedding a long complaint as a single vector blurs its distinct issues, hurting retrieval precision; 500-char chunks keep each vector topically focused, and the 50-char overlap prevents a point being cut across a boundary. This also matches the 500/50 configuration of the provided full-scale store, easing the transition to Tasks 3–4. Result: **34,978 chunks** (2.91 per complaint, mean 368 chars).

3.3 Embedding model

`sentence-transformers/all-MiniLM-L6-v2` (**384-dim**, ~80 MB). Chosen because it is fast and CPU-friendly (the full sample embedded in ~15 min on CPU), strong on short-text semantic similarity, and — critically — **identical to the model behind the provided full-scale vector store**, so Task 2 and Tasks 3–4 share one embedding space. Vectors are L2-normalised so inner product equals cosine similarity.

3.4 Vector store

A **FAISS** `IndexFlatIP` index persisted to `vector_store/index.faiss` ($34,978 \times 384$). Because FAISS stores only vectors, chunk text and per-chunk metadata are kept in a row-aligned sidecar `vector_store/chunks.parquet` (row $i \leftrightarrow$ vector i) carrying `complaint_id`, `product_category`, `product`, `issue`, `sub_issue`, `company`, `state`, `date_received`, `chunk_index`, `total_chunks` and the chunk text — so every retrieved chunk traces back to its source complaint.

FAISS is used instead of ChromaDB (both are permitted by the challenge) because ChromaDB's heavy dependency chain would not download reliably on the available network; FAISS provides the same cosine top-k retrieval with a single lightweight dependency.

3.5 Retrieval sanity check

Query *"why was i charged a late fee on my credit card"* returns highly relevant Credit Card complaints:

Rank	Category	Cosine	Excerpt
1	Credit Card	0.844	"...i logged into my credit card account to make my regular monthly payment that was due..."
2	Credit Card	0.831	"...the credit card company sent me an annual fee... now they are showing i'm 30 days late..."
3	Credit Card	0.813	"...i paid chase bank \$210.00... chase bank charged me a late fee of \$37.00..."

3.6 Task 2 deliverables

- [x] Chunking + embedding + indexing script: `src/chunk_embed_index.py`
- [x] Persisted vector store: `vector_store/index.faiss` + `vector_store/chunks.parquet`
- [x] This report section (sampling, chunking, embedding, store) + `reports/task2_summary.json`

4. Repository & Process

- **Structure** follows the prescribed layout (`src/` , `notebooks/` , `tests/` , `vector_store/` , `.github/workflows/` , `.vscode/`).

- **CI:** `.github/workflows/unittests.yml` runs the unit tests in `tests/`.
- **Version control:** work is on `main` at <https://github.com/yomiyu15/rag-complaint-chatbot>.
- **Data hygiene:** large data (`data/*.csv` , `*.zip` , `*.parquet`) and the vector store are git-ignored and regenerated from code.

5. Next Steps (final submission)

1. **Task 3** — Load the pre-built full-scale vector store, implement the retriever (top-k=5) + prompt template + LLM generator, and produce the qualitative evaluation table (Question / Answer / Sources / Score / Comments).
2. **Task 4** — Build the Gradio/Streamlit chat UI with source display and a Clear button; add screenshots to the final report.
3. Convert this into the polished Medium-style final report.